ABRA — the model family (living reference)

Version 3.30.0 · Last updated 2026-07-31.

The single source of truth for what each model **is**, **how it works**, its **honest current status**, and **where the code lives**.

> ***How this file went wrong, recorded because it caused real damage.*** *Between 2026-07-28 and 2026-07-30 it was not updated while ~40 commits landed, and a session then mischaracterised the whole model family from it:* ***DODUO*** *described as unbuilt when it had been built, wired, controlled and measured at 42.0%;* ***MEDICHAM*** *audited in its superseded v2 file, with that file's limitations reported as ABRA's; top-K pruning "proposed" that* `engine/fit_joint.js` *already implemented. Two rules came out of it — read the implementation and check which file a consumer actually* `require`*s, and never report a red test as a "known failure" (*`CLAUDE.md`*).*

Which files actually play a game: `engine/mew.js` loads `engine/magnemite.js` (MAG), and `engine/board.js` loads `engine/medicham2-browser.js` for damage. That is the entire live path. Everything else on this page is off it — which does **not** mean dead, only that "is this live" must be answered by grepping for consumers rather than by file date. Superseded implementations live in `engine/graveyard/` and are not to be fixed, cited, or audited as current.

Guiding principle: **garbage in, garbage out.** The browser engine's **damage math is now validated** against the Smogon damage calculator (within 5% on 100% of tested scenarios — see MEDICHAM below). The remaining GIGO caveats are the rollout *policy* and format-specific data (Champions rule changes, some Mega/ability specifics) — stated per-model.

DODUO — Doubles Optimiser: Decisions United, One turn (named 2026-07-28)

Job: score the PAIR of choices, not two choices separately. Named for the two heads on one body: two slots, one decision. 18 coordination features — `focusFireKills`, `redirectThenAttack`, `boostsPartnerDamage`, `speedSetupHelpsPartner`, `weatherSetupHelpsPartner`, `healsPartner`, `doubleKO`, `flinchThenSetup`, `screenWhileThreatened`, `spreadFreeBesideAlly`, and the rest. **Why it matters, and it is a strategic argument not a tidiness one (Will's):** a team must optimise for TEAM success, not individual Pokémon success. A policy that picks each slot independently can be set positions that REQUIRE coordination and will fail them every time — a repeatable hole rather than variance, and precisely what WOBBUFFET searches for. **I retired this earlier and was wrong.** The retirement rested on the double-target rate — MAG 24.6% against humans 23.2%. That metric touches **2 of the 18 features**. Judging a team-coordination model by a targeting statistic is judging a ninth of it. **Refitted at 48 features 2026-07-28** — 5,250 clean games, 18,740 usable joint turns, 15,345 train / 3,395 held out:

predicting which PAIR a human clicked	log-lik	top-1
two moves decided separately (what MAG did)	−3.6374	5.9%
refitted, joint terms forced to zero	−3.1643	12.0%
with the joint terms	−2.9890	**14.5%**

Read the middle row before the last. **Over half the gain is just refitting the single-move weights on pair data**; only the final 2.5 points belong to the coordination terms themselves. And all of this predicts a human click — it is not evidence the pair wins more games. **Now wired into** `magnemite.js` **for the first**

time (`--joint` , off by default). It had never once been in the loop, so the project had never tested whether coordinated choice helps; retiring it would have closed a question that was never opened. **The wiring bug worth remembering:** the first smoke test decided **0 pairs and fell back 99 times** while reporting nothing wrong — 0 games discarded, no error. The partner's options were read from the raw request rather than the reshaped list `chooseMove` receives, so every partner candidate parsed as unusable. A head-to-head run at that point would have returned ~50% and I would have reported that coordination does not help. It was caught only because the fallback is COUNTED and printed. Fixed: 100% of eligible turns now decided as a pair. **MEASURED 2026-07-28, AND IT LOSES.** Coordination ON against coordination ZEROED — the entire pair path either way, same single weights, same top-K cap, same softmax over pairs, so the only difference is the 18 coordination weights. 2,000 seed-paired games, harness fair at 49.3%:

	result
unpaired win rate, coordination ON	**42.0%** [39.9, 44.3] over 1,934 games
decisive pairs, coordination ON wins both	**28.4%** [23.9, 33.3] of 356

Not close, well powered, and consistent across every cut. It loses hardest in short games (22.0% under 8 turns) and when it does not draw first blood (14.1%), which is a bot giving away tempo. It KOs less (22.3% against 25.1%) and Protects nearly twice as often (1.74% against 0.93%).

Why, and it is the same lesson MACHAMP taught. These are IMITATION weights. The fit prices `spreadFreeBesideAlly` at −5.054, `terrainSetupHelpsPartner` at −3.989 and `screenWhileThreatened` at −3.372, at lambda = 0. Those are statements that humans rarely click those pairs, not that the pairs are bad — and a bot told to avoid a free spread move beside its own ally by −5 will decline its best plays. Predicting a human pair (14.5% top-1, up from 5.9%) and winning are different objectives, and this is the cleanest separation of the two the project has measured.

What this does NOT settle. Will's argument was about EXPLOITABILITY — that a bot choosing each slot independently can be set positions it fails every time. That is a claim about the worst case against a prepared opponent, not about the average, and a policy can be worse on average while being harder to counter. Nothing here tests it. `engine/exploit.js` now accepts `--target <weights.json>` , so WOBBUFFET can be pointed at DODUO for the first time. Until that runs, the coordination question is open, not closed.

The coordination features are not refuted either — the imitation fit of them is. Refitting the 18 pair weights for WINNING rather than for resemblance (MACHAMP over the joint vector) is the untested version of this idea.

UPDATED 2026-07-30 — this is now roadmap item 1, and the gap is exact. The evidence for it got much stronger: four knowledge additions produced four nulls that day while two objective changes produced two large wins (see MAGNEMITE). DODUO has only ever been fitted to the losing objective. The remaining work is wiring, not a new model:

- `engine/train_policy.js` has **no joint support** at all.
- `magnemite.js` 's learning gradient is sized to `this.w` (53 single weights), while the joint vector is `this.wj` (53 + **21** pair weights — the list grew from 18). So self-play training cannot reach the coordination weights.
- The pair softmax is the same conditional logit, and `accumulateLogitGrad(g, vecs, probs, j, nW)` is already generic over vector length.

Do choice-lock first. `fit_policy.js` hands `candidates()` all four sheet moves with no legality filter, so a choice-locked human appears to have had ~9 options when they had 4 — the logit denominator contains actions that were never available, on **6.52%** of items. Both DODUO arms inherit that error today.

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost **31.2%** on decisive pairs and would have been reported as "coordination does not help."

Already implemented, do not rebuild it: top-K capping by single-move score, keeping the human's chosen pair regardless of rank so the fit cannot manufacture agreement, and reporting how often the chosen pair falls outside K. `--joint-zero` is a true control (whole pair path, coordination weights zeroed) and `stats.jointFellBack` counts degradation.

Measured cost of the pair path, on a real mid-game board (2026-07-30): 9 × 8 = **72 joint actions** per side. Only **28 of 72** have a non-zero joint vector — the other 44 score exactly as the sum of two singles already computed. With two Pokémon the coordination graph is a single edge, so the MARL factorisation machinery (QMIX, QPLEX, Max-Plus) is unnecessary; what that literature does contribute is the reason independent scoring fails, since a monotonic factorisation cannot represent "Protect while my partner removes the threat." **Code:** `engine/fit_joint.js` → `data/policy-weights-joint.json`; played via `--joint` in `engine/mew.js`.

MACHAMP — Match-Arbitrated CHAMpion Promotion (named 2026-07-28)

Job: make MAG stronger by WINNING, not by resembling people. **Method:** champion/challenger hill-climb over MAG's policy weights. Candidates are perturbations of the current champion; each plays the champion over hundreds of seed-matched games and is promoted only when a Wilson interval clears 50%. The opponent is the CURRENT champion, so the bar rises every generation — hill-climbing against a frozen target produces a policy that beats that target and nothing else, which this project already fell for once. **Why it matters more than anything else on the list:** every other model here is fitted to PREDICT A HUMAN CLICK. That is a ceiling, and it is measured: re-optimising the same features for winning moved the kill proxy from +0.34 to +2.75, an eightfold change on exactly the signal the imitation fit throws away. MACHAMP is the only component whose objective is the thing actually wanted. **Honest status: half-run and stale.** The 2026-07-26 run completed **2 of 6 generations on a 17-FEATURE vector and recorded no verdict**. The vector is now 48. Re-running it is the single largest untested lever in the project. **Guards worth keeping:** every promoted champion is played against EVERY previous generation, not just the one it displaced — this metagame is cyclic, so "gen 5 beats gen 4" does not establish progress, and a cycle would otherwise look like improvement forever. **Limit, stated:** it searches the weight vector, not a policy space. It cannot learn anything the feature set cannot see, and after 2026-07-28 we know the feature set is the binding constraint for a static model. **KEPT, 2026-07-30** (Will, reversing an earlier call to graveyard it). The **artifact** is stale — `data/policy-weights-machamp.json` is a **48-feature** vector against today's 53, trained under the broken mega handling — but the **method is alive and has a successor**: `engine/train_policy.js` implements the same win-objective idea by policy gradient over self-play, and is the thing that measured 55.9%. Re-running MACHAMP on the current vector is roadmap item 4. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, because this metagame is cyclic and "gen 5 beats gen 4" is not progress. **Code:** `engine/ladder.js` → `data/ladder.json`. Companion: `engine/brood.js` (how many candidates a generation can actually tell apart).

> ***RECONCILED 2026-07-31.*** *That 55.9% was measured on the* ***53-feature vector with switching OFF****. Repeating the experiment on the* ***56-feature vector with switching ON*** *gives* ***48.1%*** *[46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration*

> *being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

WOBBUFFET — the counter that finds MAG's leak (named 2026-07-28)

Job: how readable is MAG? Build the bot whose only purpose is to beat it, and see how badly it wins. **Method:** hill-climb over MAG's OWN feature weights, maximising win rate against MAG. Named for Counter/Mirror Coat — whatever you do, it returns the thing that beats it. **Honest status: stale, and its result is the most important number in the repo.** On the 17-feature vector a counter found in forty minutes beat MAG **63.2%** [56.6, 69.3], with a mirror control at 47.5%. That challenger was not a counter in the rock-paper-scissors sense — it was simply a better player, drawn from the same features and optimised for wins instead of resemblance. Two feature-generations old. **Read it with MACHAMP:** MACHAMP raises the bar, WOBBUFFET measures how easily the bar is cleared. Together they are the win-objective loop; separately neither means much. **Caveat on the metric itself (2026-07-28):** exploitability grades "can a prepared opponent read us", which assumes an adversary that studies you over many games. A tournament opponent has never seen you play. It is a real number and it is not the same as "do we win", which has never been measured against a human at all. **BACK ON THE ACTIVE LIST, 2026-07-30** (Will: *"ADD WOBBUFFET BACK TO THE LIST"*). Now **three** feature-generations stale (17 → 53), and it is roadmap item 3 because it is the only measurement here that is not bots grading bots on average — it grades *readability*. A policy can improve on average and stay exactly as exploitable; those are different numbers and only one of them has been moving. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, which is the only way to test the exploitability argument the 42.0% result explicitly does **not** settle. **Code:** `engine/exploit.js` → `data/exploitability.json`.

JOLTEON — Joint Odds, Ladder-Trained Expected-Outcome Network

Job: instant pre-game win probability from two team sheets. **Method:** Bradley-Terry-style logistic — sum of learned per-species strengths + speed/firepower edges + a team-vs-team type-coverage term, through a sigmoid. Rarity-aware L2 shrinkage; recency-weighted. **Honest status: demoted to a fast prior, not an oracle.** Held-out backtest (`eval_harness.py`): accuracy ~55% (at the format's skill ceiling) but **log-loss ties a coin** (0.699 vs 0.693) and only matches player-Elo. Overconfident (temperature ≈ 6 to calibrate). The team sheet simply doesn't determine the winner much in a non-transitive, high-variance format — that's a real finding, not just a weak model. **Code:** `engine/jolteon.py` (train), `pwin()` in `web/index.html` (deployed). **RETIREMENT WITHDRAWN 2026-07-30 — it was retrained and it works.** The old verdict below was reached on a model trained through the raw ladder store, where `jolteon.py` defined its own `load_games()` and read a population that is ~87% bots, forfeits and stubs. Retrained on **self-play** (`JOLTEON_SELF` ; `JOLTEON_RAW=1` restores the old behaviour), it moved from worse-than-a-coin to genuinely predictive. It is a real input to DITTO again rather than a candidate for deletion.

But it is ADDITIVE, and that is DITTO's binding problem — 258 species weights plus 2 extras (speed, damage), with no pairwise term. Measured 2026-07-30: the additive species block can move the score by ~6.3 while speed and damage together move it by at most ~0.4, so hill-climbing it converges on the six highest-weighted species. **It cannot represent that Pelipper + Archaludon is worth more than Pelipper plus Archaludon** — the same expressiveness failure as DODUO, one level up. See DITTO.

The old verdict, kept because a prior conclusion is never silently rewritten. Every preview-level model in this project sat at the same coin-flip ceiling — JOLTEON, preview roles, CHOMP-EV, and as of 3.2.0 WAR. The 2026-07-25 finding that the 55% skill ceiling was itself bot-contaminated (52.4% clean, CI includes 50) makes the ceiling lower than JOLTEON was built against, not higher. A low-rank non-transitive term would not change that. It survives only as a candidate shortlister for DITTO; it should make no win-probability claim anywhere.

MEDICHAM — Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves

Job: grounded win rate by actually playing the matchup out. **Method:** real Gen-9 **doubles** Monte-Carlo rollout (`engine/medicham2-browser.js` , embedded as `MEDI2` in the site). Damage formula with boosts, spread ×0.75, crit, rolls, STAB, type, weather, Trick Room, Tailwind, priority, Protect, items (scarf/band/specs/AV/Life Orb/leftovers/sitrus/**Expert Belt/Muscle Band/Wise Glasses**), and a **validated ability/item layer** (Ruin quartet, Solar Power, Guts, Orichalcum Pulse, Hadron Engine, Adaptability, Technician, Tinted Lens, Filter/Solid Rock, Multiscale, Thick Fat, Heatproof, Purifying Salt, type-immunity abilities). **Mega abilities tracked** (base vs Mega stone: Staraptor→Contrary, Swampert→Swift Swim, + canonical Megas). Status, Fake Out flinch, **recoil**, **self-stat-drop moves with Contrary flip**, **weather-speed abilities** (Swift Swim etc.). Policy = **behaviour cloning** (samples the move real players click) + take an obvious KO + need-based Protect, now **accuracy-weighted** (a 70% nuke isn't a guaranteed KO) and recoil-aware (reduces the fast-frail over-crediting). **Win% backtest — the hard finding + the twist (2026-07-23):** on 600+ held-out real games, MEDICHAM's raw P(win) **does not beat a coin** (log-loss 1.2 vs 0.69) and picks the actual winner only **~44% of decisive calls — below chance**. Below-chance is not "no signal": it means the win% is **systematically inverted** (the policy backs the fast/offensive team; that team loses more — the Staraptor bias, quantified). Held-out Platt recalibration comes out with a **negative slope** and just edges the coin (0.6897 vs 0.6931) — real but *tiny*, because even **player-Elo ≈ coin (0.687)** here: Champions is near-unpredictable at the game level from sheets alone. **Consequences:** (1) the win% is a matchup heuristic, not a game predictor; (2) **DITTO was optimising a backwards signal** — building teams the biased engine loves (confirmed) — so its objective must be de-biased/flipped before "best team" means anything; (3) the durable value is the **validated damage** (exact against the Smogon damage calculator → CHOMP/ORB), which is genuinely not a coin. Harness: `engine/backtest_winrate.js` , report `data/winrate-backtest.json` . **Policy validation (2026-07-23):** the behaviour-clone (the policy's backbone) predicts held-out human moves at **top-1 35.9% (CI 35.2–36.5), top-3 71.6%**, cross-entropy 2.27 nats — beating the species-agnostic baseline (4.54) and uniform-over-moveset (2.91), so the priors carry real signal, but human move choice has genuine entropy (the clone is a *modest* predictor). A phase-conditioning improvement was tried and did **not** beat the proper score, so it wasn't shipped. This is a conservative lower bound on the full policy (the KO-take/Protect overrides only raise agreement on those turns). Harness: `engine/eval_policy.py` , report `data/policy-eval.json` . **So MEDICHAM's win rate is** `P(win | realistic cloned play)` **, now with the clone's fidelity measured — not** `P(win)` **ground-truthed. Honest status:** big improvement over the old 1v1 chain (which gave 0%/100%). Mirror 0.50, healthy spread, 400 rollouts in ~30ms, results carry a 95% CI on the site. **The damage math is now VALIDATED** against the Smogon damage calculator (MIT ground truth): with stats aligned, MEDICHAM matches the calc to the integer on 18/22 meta scenarios; after adding the Ruin quartet + Solar Power + Guts, it's **within 5% on 100% of scenarios, median error 0%** (worst 3% = 16-roll rounding). See `engine/validate_damage.js` and `data/damage-validation.json` . The remaining caveat is the *policy* (behaviour-cloned; over-credits speed control), not the damage numbers. **THERE IS ONLY ONE MEDICHAM NOW (2026-07-30).** Will: *"LETS JUST CALL THE FUNCTIONAL MEDICHAM MEDICHAM, NO NEED FOR V3, FOLD OLD DEAD VERSIONS INTO A GRAVEYARD."* `engine/medicham.js`

was the **v2** singles rollout — 1v1 in a doubles format, a hardcoded 14-move priority list, unseeded `Math.random`, no team sheets, no megas. It is now `engine/graveyard/medicham-v2-singles.js` and **must not be read, fixed, or cited as MEDICHAM's state**; a session audited it by mistake and reported its limitations as ABRA's. Its two consumers were repointed at the doubles engine (`engine/ditto.js`, `tests/test-medicham.js`, the latter now guarding the live engine's three invariants — mirror symmetry 0.538, range, antisymmetry).

What MAG borrows from it: only `buildMon` and `dmgRange`. MAG never runs the rollout — so describing MEDICHAM as "MAG's damage calculator" is describing the borrowed half, not the model. **Code:** `engine/medicham2-browser.js`; `mcWinProb` / `mcWinProbI` in the site delegate to it (now Laplace-smoothed so a rollout can never read 0%/100%). Abilities patched from a curated meta map; move names + items from real ladder sets. **Open: Champions rule changes vs Gen 9 (sleep, paralysis, specific moves) are NOT yet modelled — pending the exact format rules** (flagged rather than guessed). Also: DAgger/improve the rollout policy; Protosynthesis/Quark Drive stat boosts; Mega stat/type swaps (abilities are done, stats aren't). Validation harness: `engine/validate_damage.js`, report `data/damage-validation.json`.

DITTO — Double-oracle Iterative Team-Tuning Optimiser

Job: turn your seed team into the best version against the live meta. **Method (as of 2026-07-23):** (1) solves the **Nash equilibrium** over the archetype match-up matrix (`data/meta-nash.json`: Rain/Sand/FakeOut), (2) **best-responds to that equilibrium**, and — the key fix — the hill-climb now optimises the **grounded MEDICHAM value**, using JOLTEON only to shortlist candidates. Enforces the **item clause** (one item per team). (3) reports a per-archetype **matchup bar chart** ("how your team does vs the meta") and names your exploiters. **Two modes (as of 2026-07-23):** *Refine my team* (keep your core, only the highest-impact swaps that clear +5%) and *Build a perfect team* (full hill-climb). Both score against **all data-derived archetypes** (see below) with a weight floor so every threat counts, show an all-archetype matchup bar chart, and use accuracy-weighted, recoil-aware MEDICHAM as the objective. **Honest status:** optimises the **now-validated** MEDICHAM damage engine (was JOLTEON≈coin, which produced junk). The remaining caveat is the rollout *policy*, not the damage. Win-rate bars are Laplace-smoothed (never 0%/100%). **Code:** `runDitto(mode)` in `web/index.html`; archetypes from `engine/archetypes.py`; equilibrium math `engine/slowking/nash.py`.

STATUS 2026-07-30 — PARKED, WITH A NAMED FIRST STEP. Will: *"I WOULD LIKE TO IMPLEMENT DITTO AT SOME POINT."* Nothing currently `require`s `engine/ditto.js`, which reads as dead; it is not. It is a goal with known defects, and it was rebuilt-in-part this session:

Fixed. Its referee was v2's `winProb` — a **1v1 sequential-singles** rollout being used to judge four-Pokemon doubles teams. The "grounded rollout" meant to catch JOLTEON Goodharting itself had no spread damage, no redirection, no Protect and no positioning. Repointed at `winProb2`, the doubles engine; verified running (`medichamRank` returns 0.517 rather than null).

Still broken, and the rebuild Will approved:

1. **The objective is ~94% "add up six numbers."** It hill-climbs JOLTEON, which is additive (see above), so it converges on the top-6 by weight. **No synergy is representable** — no weather core, no redirection-plus-setup, no Trick Room pairing.
2. **The screen decides what the referee ever sees.** The coarse-to-fine design is sound — JOLTEON screens thousands, MEDICHAM re-ranks finalists — but MEDICHAM only evaluates what JOLTEON proposed. Fixing the referee buys little while the screen cannot *propose* a synergy team.
3. `coverage()` **does not measure coverage.** It reports "win rate vs teams that run Basculegion", but that is the same additive score filtered by which teams contain it. It cannot detect whether you have an

answer — only how strong the teams that run it happen to be. The threat penalty is built on top of that.

4. **Unseeded** `Math.random()` in `loadMeta`'s shuffle, so the gauntlet and therefore the chosen team differ every run.
5. **It scores the six, not the four you bring.** `mean(six.map(spd))` averages all six; VGC is the bring.

The planned fix: pairwise terms over **roles**, not species — 258 species is ~33,000 pairs, far too many to fit on ~6,000 games, while `roles.py` tags 344 species into 52 roles. Same shape as DODUO's pair block, and subject to the same warning: fitted for resemblance it will lose. **Prerequisite done 2026-07-30:** the role vocabulary could not see megas at all, which is fatal here because the weather and terrain cores *are* megas. See ROLES.

KADABRA — Key Analysis of Decisions, Advice & Better Replay Annotation

Job: coach a real replay — take you to the turns that mattered. **Method (as of 2026-07-23):** parses the replay log to find decisive turns (KOs, losses), then runs a **clean move-by-move walkthrough** — big prev/next arrows, sprites + HP each key turn, and a bold **"what you should've done"** panel with the prescriptive fix. No Showdown iframe (dropped as clutter). **Works offline (`file://`)**: coaches from a locally-bundled set of recent games (`data/kad-replays.js`), with a recent-games picker and a raw-log paste fallback. **PORY win% wired in (2026-07-23) — RETRACTED 2026-07-25.** PORY is not a validated value net: its fitted weights reduce to `sigmoid(1.256*alive_diff + 1.544*hp_diff)`, it ties that two-feature material baseline exactly, and `turn` is structurally pinned to zero. The chip still renders, but it reports material arithmetic, not a learned value. Original entry follows. Each key moment shows a **"you're at X%" chip** from PORY (`poryWin()` reads `data/pory.js`; features = mons alive out of 4 + mean active HP + turn). This is a *validated* per-turn readout (PORY: log-loss 0.567 vs coin 0.693, calibrated), unlike the still-heuristic prose fix. **Honest status:** working offline; the prescriptive text is heuristic ("you traded X for nothing — Protect/pivot keeps it alive"), not yet equilibrium-grade — that's ALAKAZAM, later — but the win% chip beside it is real. **Code:** `runKadabra`, `kadCoach`, `kadBuild`, `renderKad`, `poryWin` in `web/index.html`; bundle from `engine/refresh-site-data.py`. **Open:** deeper per-turn analysis (win-prob delta per decision) once the engine + value net are wired.

SLOWKING — Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver

Job: the endgame — tell you the equilibrium-best move (and win %) on a live position. **Method:** the poker-AI stack (CFR → DeepStack → Libratus → ReBeL) adapted to VGC. `engine/slowking/`: `nash.py` (equilibrium, verified on RPS/2×2), `belief.py` (public-belief-state + Bayesian filter), `ismcts.py` (simultaneous-move regret matching, recovers exact Nash), `game.py` (engine interface), `solver.py` (team-preview Nash + continual re-solve → bring **mix** + win%), `value.py` (learned leaf evaluator). **Preview-Nash built + evaluated (2026-07-23):** `engine/slowking_preview.py` solves GURU's archetype matchup matrix (game count generated into `data/live.js`; the old hardcoded size is retracted, S13) to an equilibrium mixed strategy and grades it by **exploitability** (the spec's bar). Result → `data/slowking-eval.json` (+ `data/slowking.js`): the equilibrium is **Kingambit-Basculegion 0.84 / Garchomp-Incineroar 0.16**; exploitability **Nash ≈ 0 vs uniform 0.109** (mixing over the *right* decks is far less exploitable than spreading evenly). Greedy "pick the single best deck" ≈ Nash **here** because this meta is currently near-transitive at the coarse archetype level (a dominant deck) — an honest finding, not a win for mixing. **But a real non-transitive cycle exists** (Charizard-Venusaur → Whimsicott-Garchomp → Garchomp-Incineroar → back, ~0.10 edge each leg), so greedy is *not* universally safe; finer, playstyle-level archetypes (stall / Trick

Room / perish-trap / setup) would expose more cycles. CI propagates matchup-count uncertainty (Beta resampling); the greedy-vs-Nash gap CI upper bound is 0.27, i.e. under plausible resamples the meta *is* non-transitive. Test: `tests/test-slowking.py` (RPS hand-check + shipped-artifact invariants), gated in CI. **Honest status:** preview-Nash is now a **real, tested model on real data** (exploitability + baseline + CI), not just a chassis. The in-battle search (IS-MCTS/PIMC → ReBeL) is still a rung below target and not yet wired to the engine — that's ALAKAZAM. **Code:** `engine/slowking_preview.py`, `engine/slowking/*`; white paper `docs/POKER-TO-POKEMON.md`. **Open:** wire `ChampionsGame` to the real engine; PIMC → outcome-sampling MCCFR / PBS re-solving; train the value net via self-play.

The learning core (the flywheel)

value net: `engine/train_value.py` reconstructs per-turn HP state and regresses the outcome → `data/value-net.json`. Beats a coin (log-loss 0.682), calibrated, compressed at the tails (thin data). It's SLOWKING's leaf evaluator. **self-play:** `sim/generate-dataset.js` writes engine games into the store schema (unlimited, unbiased data). **flywheel:** `engine/flywheel.py` — self-play → retrain → re-evaluate → report the delta. The thing that makes ABRA *learn over time*. **live data + auto-refresh (2026-07-23):** `engine/durable-ingest.js` pulls new ladder replays; a **daily scheduled task** runs it, then `engine/refresh-site-data.py` regenerates `data/archetypes.json` (via `engine/archetypes.py` — archetypes *discovered* from the games by k-means, not hand-listed), `data/live.js` (counts + archetypes the site loads live) and `data/kad-replays.js` (offline KADABRA bundle). So the site's numbers and meta **grow on their own**.

CHOMP / ORB (companion tools, separate repo)

CHOMP — the bring-4/lead-2 decision engine (Showdown userscript). Picks your best 4 and 2 leads by exact damage over the opponent's whole six. *Open:* bring/lead should be a minimax matrix game (`nash.py`), not greedy coverage. **CHOMP-EV proof (2026-07-23) — honest NULL.** The winnable test (do CHOMP's brings beat humans' actual brings on held-out games?) was run over 1,205 human games (`engine/chomp_ev.js` → `data/chomp-ev.json`). CHOMP's damage-coverage bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), **ties** an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). Robust to dropping all forfeits; a measured selection audit shows the mild eval-set bias *favors* CHOMP, so the null is conservative. **The bring decision sits at the same near-coin ceiling as pre-game prediction** — CHOMP's damage math stays validated and useful as a calculator/EV display, but "CHOMP builds better brings" is not yet empirically supported. This is the guardrail that stops a DITTO-style on a signal-free metric. **Belief-weighting tried (2026-07-23):** scoring coverage vs the opponent's *likely 4* (top-bringRate mons) instead of all 6 also ties the coin (log-loss 0.6924 vs 0.6931), so belief-awareness alone doesn't rescue it — the bring decision is at the format ceiling, full stop. *Next (if pursued):* full XATU-set belief + SLOWKING lead stage-game + PORY leaf value, then re-run this exact harness and measure the lift. **ORB** — On-battle Read Board, the damage calculator. **Decision (2026-07-23):** rather than fork the Smogon calc (the hosted one can't be auto-filled cross-origin, and a fork needs a build), ORB is a **validated Smogon-grade substitute built into the CHOMP dock** — same damage engine validated against the Smogon damage calculator, reading the **live battle**: real stats/items, boosts on both actives (Intimidate/setup), weather, terrain, Helping Hand, spread, screens; it prints the conditions it applied. One-click install, auto-updates. (`docs/ORB-smogon-fork.md` kept as the record of the fork option we chose against.)

Evaluation & honesty (cross-cutting)

- `engine/eval_harness.py` — held-out log-loss / Brier / calibration vs coin, Elo, usage baselines with bootstrap CIs. **The bar every model must clear.**
- `engine/calibrate.py` — temperature scaling.
- `docs/THESIS-REVIEW.md` / `THESIS-REVIEW-v2.md` — strict self-critique with fixes (willing to scrap/rebuild).
- `docs/COMPETITORS.md` — VGC-Bench, PokéLLMon, offline-RL transformers, and how we refine them.
- ~~Non-transitivity: `data/nontransitivity.json` — the meta is rock-paper-scissors.~~ **WITHDRAWN 2026-07-26.** The file was computed 2026-07-23, two days before the quality filter, and nothing regenerated it — so the cycles it showed were measured over a corpus that is 87% bots, forfeits and stubs. Re-run on clean data, SLOWKING's equilibrium collapses to **100% on a single option** with **zero** gap between mixing and picking the best, and the clean GURU matrix contains **0 decisive matchups**. The file is deleted rather than kept, because a stale artifact on disk is how a retracted claim gets quoted again. The honest reading is *no usable input*, not *mixing does not help*: a Nash solution over a matrix of noise says nothing either way. See CHANGELOG 3.16.0.

Status of the "one thing that unblocks everything"

DONE (2026-07-23): the engine's damage math is validated against the Smogon damage calculator — within 5% on 100% of 31 tested scenarios (`engine/validate_damage.js` , `data/damage-validation.json`). MEDICHAM/DITTO no longer rest on unverified numbers. **Next priorities, in order:** (1) get the **Champions rule changes** (sleep, paralysis, moves) from the format and model them — the current biggest data gap; (2) harden the rollout **policy** (the last GIGO lever); (3) grow the dataset via the daily pull + self-play so the discovered archetypes and win rates sharpen.

ROLES — multi-label team composition (added 2026-07-24)

Job: describe every team by the set of functional roles it reveals, instead of forcing one archetype label. **Method:** 26 roles (speed control, weather, terrain, disruption, status/debuff, priority, prankster, setup, healing, screens, walls, pivot, trapping, perish, ally-support, item-disruption, physical/special attacker). Each **species earns a role from data** — credited once it is observed doing it (≥2 times). Multi-effect moves carry several *factual* roles (Matcha Gotcha = attack+heal+status; Body Press = wall+attack; Knock Off = attack+item-strip; Fake Out = tempo, not attacker). Role *presence* is binary and data-justified; graded strength is **not** hand-set — it is the learned output of the NMF (see below). Team role vectors are built from the **team-preview six** (leak-free). Outputs a **role-pair matchup matrix** with Wilson CIs. **Honest status / result:** the role-pair matrix **pools the data** — median cell **n=20** across 1,051 cells (2026-07-25, clean games) vs the old single-label archetype cells of n=11–18. Pooling still wins, but only just — and the 7,971 published in v2.6.0 was retracted in 2.7.0 as an over-tagging artifact. But predicting the winner from **preview roles ties a coin** (held-out log-loss 0.694 vs 0.693) — consistent with the sheet-level null. The value is descriptive + attribution, not prediction. Per-role logistic coefficients give **win-credit per role**; KO-credit per species comes from the turn log. **CAPABILITY LAYER, added 2026-07-30 — because team preview never shows a mega.** Will: *"ROLES ARTIFACTS IS ALSO VERY OLD" / "WE HAVE SMOGON DATA."* The artifact held 280 tagged species and **zero mega formes**, while megas are 26.0% of this format's usage. The failure split cleanly: `tyranitar` weather_sand 95.2%, `pelipper` weather_rain 97.9% and `torkoal` weather_sun 94.4% all worked, because those abilities sit on the base forme; `raichu` had no `terrain_electric` despite Raichu-Mega-X being the format's only Electric Surge, and `charizard` and `swampert` were **absent entirely**.

It was not a bug in this file. Checked against the store directly: of **58,920** team-preview species names in clean games, **zero** are mega formes (the only `/mega/` match is Meganium). That is correct Pokémon behaviour — preview shows the base and the mega is revealed only on evolution — so no amount of regenerating could produce them.

Fix: a `capability` block derived from `data/smogon-priors.json`, which carries mega formes as first-class rows *with* their abilities, over the whole ladder instead of a few thousand replays. Mega rows are folded onto the BASE name, read from `mega-dex-official.json`'s `base_species` rather than by stripping the name, because teams are built from base names.

Kept separate on purpose. `roles` remains p(role | species appears), measured with Wilson bounds; `capability` is a **reachability set** — can this species play this role at all. Merging a presence flag into a measured distribution would have quietly corrupted it. Presence, not rate, is load-bearing here: Smogon records the mega rows' ability slot unreliably (Raichu-Mega-X reads "No Ability 81%, Electric Surge 19%"), so a frequency filter would discard the exact capability sought.

charizard -> weather_sun, abuser_sun from charizardmegay (Drought + Weather Ball + Solar Beam) raichu -> terrain_electric from raichumegax swampert -> abuser_rain from swampertmega

Noise checked rather than assumed: 17 species reach `weather_sun`, which looked loose until verified — Whimsicott runs Sunny Day at **16.2%**, Liepard 12.3%, Sableye 6.8%, Klefki 5.2% (Prankster sun is a real archetype), while Torkoal and Charizard-Mega-Y show *no* Sunny Day because they set it by ability. Both routes captured, neither invented.

Regenerated on 4,910 clean games (was 2,653), **344 species tagged** (was 280), 200 with a capability set, 1,101 matchup cells, median n=69. **The null holds:** held-out log-loss `roles=0.6933` against a coin's `0.6931`, CI (0.6880, 0.6981), accuracy 0.508 — role-level winner prediction still ties a coin on 1.85× the data. The capability layer is team-building **vocabulary**, not prediction, and does not change that.

Known downstream staleness: `xatu-context-sets.json` and CHOMP-EV are built on this artifact and `roles` moved with the larger corpus. Regenerate in dependency order (roles → context → CHOMP-EV) before quoting either. **Code:** `engine/roles.py` → `data/pokemon-roles.json`, `data/role-matchups.json`, `data/roles-eval.json`. Tests: `tests/test-roles.py` (19).

WAR — Wins Above Replacement (added 2026-07-24)

Job: how many wins a species adds over a freely-available replacement, controlling for teammates and opponents. **Method:** ridge-regularized **Adjusted Plus-Minus** (basketball RAPM) — logistic regression of game outcome on the difference of team-preview species-indicator vectors. Replacement = 20th-percentile β; WAR = 0.25·(β−β_repl)·games (the logistic wins conversion). Ridge shrinks rare species toward zero.
Honest status / result — WITHDRAWN 2026-07-25. WAR does not beat a coin on clean data. The previous entry read: "the species model **beats a coin** (held-out log-loss 0.6875 < 0.6931) and beats the rating baseline (0.6905) — so *which specific species* you bring at preview carries a small real signal that roles and raw sheets do not." That was measured on the **unfiltered** store.

Run both ways on the same store (v3.2.0):

	held-out log-loss	**vs coin 0.6931**	**accuracy**
unfiltered, 8,356 games	0.6860	beats it	0.539
clean, 1,061 games	**0.7048**	**worse than a coin**	**0.502**

The mechanism is visible in the coefficients: Basculegion's WAR falls from **281.87 to 23.64**, and Basculegion is one of the six Pokemon four undetected bot accounts played in 1,446 identical games. The model was learning which species belonged to the highest-volume account, not which species win. Charizard — also on

that team — is the largest negative in both runs, the same artifact inverted.

Current status: a null. Preview species composition sits at the same coin-flip ceiling as preview roles and raw sheets. WAR magnitudes are retained as a descriptive ordering only and must not be quoted as evidence that species choice predicts outcomes. **Code:** `engine/war.py` → `data/war.json`.

NMF — emergent roles / archetypes (added 2026-07-24)

Job: discover roles and archetypes from the data instead of hand-declaring them. **Method:** Non-negative Matrix Factorization (Lee & Seung 1999; Label Distribution Learning framing, Geng 2016). Two cuts: (1) team×MOVE usage (usage-weighted, so the closed-sheet censoring skew is down-weighted) → offensive cores; (2) team×ROLE → **emergent archetypes**. A team is a non-negative *blend* of factors, never one hard label; a move's loading on a role is **learned, not typed** — this is where graded primary/secondary strength legitimately comes from. **Honest status / result:** role-level factorization is the clean cut — recon-err **0.53**, six interpretable archetypes (Intimidate+Fake-Out control; physical offense; special offense+sustain; **bulky wall+screens+redirection**; Tailwind+Encore; priority). Move-level is coarser (recon-err 0.79; attacking moves dominate). Rank and the human names are the only non-data choices; rigorous rank/weighting selection by **topic coherence** (Mimno 2011) is the noted next refinement — reconstruction error is not comparable across weightings. **Code:** `engine/nmf_roles.py` → `data/nmf-roles.json`, `data/nmf.js`. Vocabulary census: `engine/vocab.py` → `data/vocab-usage.json` (tags every move/ability/item, counts real battle usage; curated roles cover 90.4% of non-neutral usage). Site booth: the **Role Foundry** (Smeargle) in `web/index.html`.

COUNTERPLAY — does the field tech for the top threats? (added 2026-07-24)

Job: measure whether players spend spare move slots answering the metagame, rather than on their own gameplan. **Method:** cross-sectional, by necessity — the store spans 3 days, so the natural temporal test ("does Fighting usage rise AFTER Kingambit rises?") has no identifying variation and was not run. Instead: for each species, compare the meta-weighted type coverage of its RARE moves (a tech slot, ≤12% of its sets) against its STANDARD kit (≥30%), where coverage weights the real 18×18 type chart by each threat's current prevalence. Paired within species, bootstrap CI over species. **Result:** tech slots carry **+0.0386** more meta-weighted coverage than standard kit, **95% CI [0.0155, 0.0617] — excludes zero**, positive in 90/148 species. Concretely, vs **Kingambit** (Dark/Steel, 29% of teams) the top tech answers are Incineroar's Close Combat (Fighting, **4×**, 127 uses) and Blastoise's Aura Sphere (4×, 90) — the "rogue Fighting coverage for Gambit" pattern, measured. **Code:** `engine/counterplay.py` → `data/counterplay.json`.

MEGA DEX — the formes the engine could not see (added 2026-07-24)

Job: give the damage engine real mega stats, types and abilities. **The gap:** the engine dex held ONE mega forme while Charizard-Mega-Y alone appears in ~906 sets, so every mega calculation silently used base-form stats. Separately, the ingest never parsed mega evolution, leaving 904/906 Charizard-Mega-Y sets with a blank ability. **Source:** Showdown's own `pokedex.json`, the data the server runs this format on. Champions invents megas that do not exist in mainline (Raichu-Mega-X/Y, Glimmora-Mega), so memory or a canonical dex is not a valid source. **Honest limit:** a mega's ability can NEVER be read from replay logs — mega evolution announces nothing. Log harvesting is retained only to discover which formes exist. Level-50 stats use an assumed competitive spread and are labelled as an approximation, since closed sheets never reveal real

EVs. **Result:** 67 mega formes in the engine dex; damage validation unchanged at 100% within 5%. **Code:** `engine/mega_harvest.js` (discovery), `engine/build_mega_dex.js` (official), `engine/merge_mega_into_engine.js` (merge).

ILLUSION — catching Zoroark-Hisui in disguise (added 2026-07-24)

Job: detect when a Pokémon on screen is actually Zoroark wearing its name. **Method:** Illusion copies the name, not the moveset. If the apparent species cannot legally learn a move and Zoroark can, the disguise is *proven* — a legality contradiction, not a probability. Learnsets from Showdown, walked through prevo/base chains; species with no learnset data are skipped rather than guessed at. **Result:** on 395 Zoroark team-sides, **156 proven disguises** (0.39 per side). Most common disguise Whimsicott (16); the moves that give it away are Hyper Voice (32) and **Bitter Malice (30)**. A floor, not an estimate — a Zoroark that only clicks shared coverage is invisible to this test. **Why it matters:** on ~1.4% of teams the Pokémon you are planning against may not be that Pokémon, and the reveal is a large sudden information gain. This is XATU's problem, not a static dex's. **Code:** `engine/illusion.js` → `data/illusion.json`.

XATU (belief state) — what the opponent could still be (added 2026-07-24)

Job: replace a fixed usage table with a per-slot *information state* that narrows as the opponent proves things. **The distinction:** "Kingambit usually runs Sucker Punch" is a statement about the population, and it never changes during a game. But in a closed-sheet format nothing is known until it is proven, and every reveal is information. The cheapest and hardest constraint is one a usage table cannot express: **a Pokémon has exactly four moves.** Once four are revealed the set is closed, and every other move in the usage table — however popular — is impossible. There the prior is not merely imprecise, it is wrong. **Method:** prior P(move | species) fitted on TRAIN games only; on held-out games we walk the turns in order and predict each move before seeing it, using only what that Pokémon revealed earlier in that same game. Scored by cross-entropy against two baselines (uniform, and the usage prior). The "already revealed" boost is **measured on train data**, not chosen: P(next move is one already seen) = 0.558, and the boost is its odds form, 1.26. An earlier draft asserted 3.0, which flattered the model — the measured value is smaller and so is the gain. **Result:** cross-entropy **1.824 vs the usage prior's 1.863** (uniform floor 6.06), top-1 **35.7% vs 34.8%**. Improvement **0.028, 95% CI [0.024, 0.031]** clustered by game — clears zero. On the 2.4% of events where all four moves are already known, belief **1.695 vs prior 2.219**: that is the four-move cap doing the work. **Honest scope:** this is the move slot only. Items and abilities are unknown-until-proven in the same way and are tracked as possibility sets but not yet scored. **EVs are different in kind** — a damage roll bounds an attacking stat to an interval and moving first proves only an inequality, so an EV spread never collapses to a value the way an ability or item does. That needs a separate interval estimator. **Code:** `engine/xatu_belief.py` → `data/xatu-belief.json`. Tests: `tests/test-xatu-belief.py` (14, incl. the uniform floor derived by hand as ln(V) and the boost re-derived from its own probability).

XATU (team context) — the belief available at team preview (added 2026-07-24)

Job: predict an opponent's set before a single turn is played, using the only information that exists at preview — the other five Pokémon. **Why it was needed:** the belief-state tracker only sharpens once something is revealed, but CHOMP decides at turn zero. That looked like a dead end for feeding better beliefs into the bring decision, until the obvious point: a set is chosen to fit a *team*. Pelipper on the roster makes Swift Swim plausible; no rain setter makes it close to pointless. **Method:** P(move | species, teammate features) where the

features are public at preview — rain/sun/sand/snow setter, Trick Room, Tailwind, redirection. Each (species, context) cell is shrunk toward the species prior by n/(n+K) with K=12, so a context seen once carries under 8% weight and cannot manufacture a signal. Fitted on train games; scored on each Pokémon's FIRST revealed move in held-out games; clustered by game. **Result:** cross-entropy **2.4415 vs the bare prior's 2.5257**, top-1 **43.0% vs 40.9%**, improvement **+0.084, 95% CI [0.074, 0.094]**. That is a *larger* gain than the in-game belief tracker (+0.028) — and unlike that one, it is available exactly where CHOMP needs it. **A domain claim, checked:** "Basculegion is Swift Swim on rain and Adaptability otherwise." Its first move, with a rain setter on the team vs without: Wave Crash **46% vs 27%**, Last Respects **25% vs 48%**, Flip Turn 19% vs 8%. Roughly a 2x swing in both directions — the ability split is visible in move choice without ever observing the ability. **Code:** `engine/xatu_context.py` → `data/xatu-context.json`. Tests: `tests/test-xatu-context.py` (14, incl. re-deriving the shrinkage weight by hand).

MEW — the self-play data engine (added 2026-07-25)

Job: remove the sample-size constraint for every question that is about the GAME rather than about PEOPLE. **Why:** 1,124 clean games can only detect an edge of ~4.2 accuracy points over a coin; a 2-point effect needs ~4,900, and human replays arrive at ~330 clean games/day. Every preview-level null in this project sits inside that blind spot. **Method:** plays the **official** Showdown Champions engine (pinned `20ad99f`) against itself on **real six-Pokémon teams sampled from the clean store** — 1,257 distinct teams, sampled by distinct team rather than by game so a bot's team contributes once. Unrevealed set slots are filled from Smogon's official moveset statistics. Battle logs go through the SAME `extract()` as a downloaded replay, so self-play records are identical in shape and every downstream reader works unchanged. **Honest status:** built and gated. 1,000 games, 0 discarded. `engine/validate_selfplay.js` runs 13 checks — store shape (S7), set realism, determinism, and mirror symmetry at **51.0%, 95% CI [45.4, 56.6]** on 300 battles, so the harness has no side bias. **What it cannot do:** self-play produces **zero** evidence about people — whether players tech for the metagame, whether rating predicts, what a human will bring. Those need real games. The current batch also uses the **random** policy, which is right for matchup structure and plumbing but must **not** train a value net: a model would learn "P(win) when both players move at random". The behaviour-cloned policy is the next step, and VGC-Bench's cross-evaluation found clone-then-self-play (BCSP) is what actually wins. **Two bad batches shipped before the gate existed**, and nothing detected either: one filled every unrevealed move slot with Tackle (13% of move events), one used a flat 11/11/11/11/11/11 spread that understated Garchomp's Attack by 13%. That is why the gate exists. **Code:** `engine/mew.js` → `data/games.selfplay.jsonl` (gitignored, seed-reproducible, every record stamped `source:"selfplay"`). Viewer `web/mew.html`. Paper `docs/MEW-whitepaper.md`.

MAGNEMITE (MAG) — Move Appraisal Grounded iN Effectiveness, Matchup, Immunity and Timing Estimates (added 2026-07-26)

Job: decide a move by looking at the other side of the field, instead of by how popular the move is. **Why:** the behaviour clone answers only *what does this species usually click?* Two gaps followed and no prior-tuning could close them — super-effective moves at 9.7% against a real 21.4%, moves that outright failed at 9.7% against 2.5%. It also made every `build_lab` number a measurement of what beats **bad** play. **Method:** three files. `engine/board.js` reconstructs the state a decision was made against and turns (move, target) pairs into **53 features** (12 at 3.21.0); `engine/fit_policy.js` fits those features to real human clicks by **conditional logit** (McFadden 1974) over **6,091 clean open-sheet games and 146,910 decisions** (117,824 train / 29,086 held out); `engine/magnemite.js` plays the fitted distribution inside the official engine. `mew.js --policy score`.

Two changes to how the policy is USED beat every change to what it knows. Measured 2026-07-30: taking the best move instead of sampling is worth **+12 points raw / 79.7% of decisive pairs**, and self-play policy improvement (`engine/train_policy.js` , REINFORCE with a trust region) wins **55.9%**. Over the same period **four separate feature additions produced four measured nulls**, and an overdispersion check across teams (~1.00, against 1.169 for a known real effect) says those nulls are genuine rather than a real effect hidden by team heterogeneity. **The objective is the binding constraint, not the knowledge** — which is why DODUO's next test is a retrain rather than more features.

Facts that reached one consumer and not the next (all fixed 2026-07-30). Every integrity bug found that day had one shape. Priority blocking sat in the tag artifact read by `clickFragility` alone, so **Sucker Punch beat a Farigiraf in every rollout ever run**. The sheet's item and ability reached `switchIn` but not `switchFeatures` , the path that actually chooses the switch. A switch-in's own ability never reached the estimate at all: over 40,001 matchups, declaring `intimidate` , `drizzle` or `drought` moved the vector in **0** of them against a `levitate` control's 2,754 — so MAG weighed bringing Incineroar in against the foe's full Attack. Now 9,227 / 3,463 / 3,438. Modelling the drop alone would have been *worse than modelling neither* (Intimidate into Kingambit is +2 Attack for them), so the whole three-stage drop pipeline runs: Contrary inverts, Clear Body deletes, Inner Focus/Own Tempo/Scrappy block Intimidate by name, Guard Dog converts to +1, Mirror Armor reflects, Defiant and Competitive retaliate. All derived by calling the dex's own handlers against a recording stub — no ability or weather is named in `board.js` . **Nothing in it is asserted.** Every "this move cannot work now" test reads a dex **data field** — `move.status` , `move.sideCondition` , `move.pseudoWeather` , `move.weather` , `move.stallingMove` — against tracked state. No move is named anywhere in `board.js` , so a new regulation needs no edit (S13). The weights are estimated, never typed, and the realism report is never consulted during fitting — it is held back as the out-of-sample check, because it stops being evidence the moment it becomes the objective. **Why open team sheets:** a choice model needs the **choice set**. A normal replay reveals only moves that were *used*, so alternatives reconstructed from revelation are biased by revelation itself. Open sheets publish all four moves of all six up front. **Fit, held out by GAME** (decisions inside a game are correlated, so splitting by decision leaks): logL/decision **−1.6006**, top-1 **33.6%** — against the behaviour clone alone at −1.9302 / 27.1% and uniform at −1.7627 / 24.1%. In-sample −1.5997, so it is not memorising; weights identical at 200/300/500 iterations. **Measured out of sample, 600 seed-matched battles per policy:** super-effective **9.71% → 14.91%** (real 21.37%), failed moves **9.68% → 6.34%** (real 2.47%), immune **4.30% → 2.92%** (real 1.91%), Protect-type **21.62% → 16.71%** (real 13.87%). Both target gaps roughly halved, and 427 of 591 games survive the quality filter against the old policy's 382. **Most of the win was aiming.** `RandomPlayerAI` chooses which foe to hit with `prng.random(2)` *before* `chooseMove` is called, so the target was a coin flip however good the move choice was — and in doubles aiming is most of what "super effective" means. **It samples, it does not take the best move** — a greedy bot sails past 23.4% super-effective and is *less* human. Same argument as DEFENSE §2. **Two findings worth keeping.** The behaviour clone alone is a *worse* probabilistic model of human choice than choosing uniformly at random, and the fit weights it at only +0.25 — it is far too confident about the popular move. And the largest learned effects are not damage terms at all, they are the "this move is already dead" terms at −2.3. Reading the board is mostly about **not clicking moves that cannot work**. **56 FEATURES AS OF 3.29.0, and the three added are large.** `data/tags.json` derives 96 move tags with their parameters and `engine/tags.js` exists to load them; board.js read NONE of them, and 72 of the 96 reached no consumer at all. The symptom Will spotted: MAG scored **Tailwind and Protect identically at −1.54**, because the only things firing on a Tailwind click were `accuracy` , `isStatus` and `priorLogP` . There was no speed-control feature in the 53.

feature	fires when	weight
`speedSwing`	it flips speed order IN MY FAVOUR; zero when already faster	**+0.983** [0.933, 1.032]
`screenValue`	it halves incoming damage AND something hits hard, graded by CATEGORY	**+1.128** [1.031, 1.225]

feature	fires when	weight
`healValue`	it heals me AND I am hurt; zero at full HP	**+2.220** [2.004, 2.436]

Written as CONDITIONS rather than flags, and that is why they fired where 3.28.0's four additions measured null: a bare "this is Tailwind" cannot help a one-ply scorer, because the payoff is on later turns. What one ply CAN see is whether the condition making it worth doing is true now.

CHOICE LOCK (3.29.0). `fit_policy.js` handed `candidates()` all four sheet moves with no legality filter, so a choice-locked human appeared to have ~9 options when they had 4 — a WRONG DENOMINATOR in the conditional logit, on 6.52% of items. Live play was never affected (the request marks the rest `disabled`). After the refit, six of eight switch features clear zero, and **switches now win the argmax**: greedy play went from 222 switch events per 60 games (all forced post-KO) to 239, where before the refit `--switching` changed nothing at all.

THE OPPONENT MODEL — job 2 of ALAKAZAM, off by default (3.29.0). `incomingThreat` took a MAX, the foe's hardest available hit, and nine features are built on it. Measured: the foe's lead clicks a damaging move **52.9%** of the time and MAG assumed 100% AND assumed it was the nastiest. Now an expectation weighted by P(their action), from the same weights — `candidates` and `featuresFor` already take `side`. Across 44 boards `protectThreatened` fell 84%, `diesBeforeMoving` 78%. **The bot stops panicking.** Needs a refit before shipping, since nine features now mean something different.

Honest status / what it does NOT do — REWRITTEN 2026-07-30, because two of the four claims here had become false and were being quoted as current. It DOES now decide switches (voluntary switches score through `switchFeatures`; the post-KO replacement is scored rather than rolled) and it DOES run a real damage calculation (`board.js` calls the damage engine throughout — `koTarget`, `killIsRoll`, `diesBeforeMoving` and the switch-survival features all read it). What remains true: it has **no model of the opponent's move**, so it cannot read a Protect or bait a switch; and it is **one ply, no search**. The weights are fitted on open-sheet games, which hedge less than closed ladder play, and ~11% of clicks could not be matched to a candidate and were dropped — mostly redirection (Follow Me, Rage Powder), where the protocol records the target that was *hit*, not the one chosen. Logit also assumes independence of irrelevant alternatives, which close-substitute moves violate; see DEFENSE §6. **Corpus (as of 3.21.0):** three open-sheet sources, deduplicated by replay id, all through quality.js — `data/games.bo3.jsonl` (our own hourly scrape of `gen9championsvgc2026regmbbo3`, whose ruleset carries **Force Open Team Sheets**, so every game publishes all six sets), the ~1% of the closed ladder store where both players agreed to sheets, and the external VGC-Bench archive. 58,085 usable decisions. **Covariate shift, corrected automatically:** open-sheet TEAMS differ from closed-sheet teams by 551.9 points of total absolute species difference (`engine/corpus_shift.js`), while measured behaviour given a board differs by at most 1.49. Every refit re-estimates on a sample reweighted to the closed-sheet species mix and reports the shift **in standard errors**. Five weights move materially — `priorLogP` 10.8 SE, `bp` 6.2 SE (sign flips) — so the **reweighted vector ships**, since MEW draws its teams from the ladder store. Board-reading weights (`eff`, `immune`, `deadStatus`) do not move. **Code:** `engine/board.js`, `engine/fit_policy.js`, `engine/magnemite.js`, `engine/corpus_shift.js` → `data/policy-weights.json` (both weight vectors, standard errors, and which shipped). Six assertions in `engine/selftest.js` under "board reading".

CHAMPIONS_SIM — the official engine (ADR-001)

Job: be the rules authority, replacing a hand-written engine that was wrong in eight silent ways. **Status:** wired and **verified**. `engine/validate_damage_sim.js` runs the 31-scenario golden master through the official engine against `@smogon/calc`: **31/31 within 2%**. That clears ADR-001 migration step 3. **Why the check mattered:** it is a test of OUR WIRING, not of Showdown. ADR-001 records four engine comparisons of which three produced confident wrong numbers from mis-wiring, none of which crashed. It caught two more

on its first run — a forced maximum roll that also forced a critical hit, and the discovery that `battle.randomChance()` bypasses `battle.random()` entirely. **Speed:** 29 battles/sec/core against the hand-written engine's 3,401. Offline only; the browser must never simulate. **Code:** `engine/champions_sim.js`, pinned commit `20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4`.

SMOGON PRIORS — official population statistics (added 2026-07-25)

Job: replace three things ABRA was guessing with measurements over the whole ladder. **What it corrected:** every Pokémon had a flat `11/11/11/11/11/11` SP spread. Real Garchomp runs `Jolly 2/32/0/0/0/32` on 42% of sets, so Attack was 161 where it should be 182 — a 13% understatement of the format's most-used attacker, in every damage figure the project had produced. It also supplies items and abilities where the closed-sheet store has 69.7% and 75.5% unknown, and **P(move is ON the set)** where our behaviour-clone measures the different quantity **P(move | action)**. **Confirmed two mechanics independently:** the SP budget is 66 (97% of real spreads spend all of it) and SP is capped at **32 per stat** (92% of spreads touch it). **Limit:** aggregate. Describes the population, never a game, and cannot be joined to a replay. **Code:** `engine/fetch_smogon_stats.js` (archived monthly by CI), `engine/smogon_priors.js` → `data/smogon-priors.json`.